Le Mans Université

Licence Informatique 2ème année

Module 174UP02 Rapport de Projet

For The King

Durand Leo, Tsamarayev Massoud, Mohamed Saandi

10 avril 2026

Table des matières

1 Introduction **3**

2 Conception **3**
2.1 Présentation du jeu 4
2.2 Fonctionnalités du jeu 5

3 Organisation du projet **5**
3.1 Planning prévisionnel 5
3.2 Répartition des tâches 6
3.3 Outils techniques utilisés 6

4 Développement **7**
4.1 Génération aléatoire de la carte 7
4.2 Affichage du jeu 8
4.3 Gestion des combats 9
4.4 Gestion du menu 9
4.5 Gestion du personnage 10
4.6 Gestion de la sauvegarde 11

5 Bilan et résultats **11**

6 Bibliographie **12**

7 Annexe **13**

1 Introduction

Cette introduction présentera le sujet qui sera traité et le travail avec une présentation du plan adopté

Contexte du projet
Ce document présente le projet de développement du jeu For The King, réalisé dans le cadre de la deuxième année de licence Informatique à l'université du Mans, pendant la période de de janvier à février 2026. L'objectif principal est la conception et l'implémentation d'un jeu vidéo en langage C, en utilisant la bibliothèque graphique SDL2 pour la gestion de l'affichage et des interactions.

Objectifs techniques
Ce projet vise à mettre en pratique des concepts informatiques fondamentaux :

- **Gestion de la mémoire** : Manipulation des pointeurs et des structures en C pour éviter toute fuite mémoire.
- **Architecture logicielle** : Structuration du code pour séparer la logique de jeu (moteur) du rendu graphique (SDL2).
- **Algorithmique** : De façon plus générale, utiliser les concepts clés vus en cours en Algorithmique 2 et Algorithmique 3.

Plan du rapport
Nous commencerons par expliquer la conception du jeu, c'est-à-dire une présentation du jeu et de ses fonctionnalités. Puis dans un second temps nous expliquerons comment le projet s'est organisé, en parlant du planning prévisionnel, de la répartition des tâches ainsi que des outils techniques utilisés. Ensuite le rapport traitera du développement du jeu, et montrera comment est programmée de façon générale chaque fonctionnalité du jeu. Enfin nous dévoilerons le bilan et les résultats de ce projet. Une bibliographie et une annexe sont également disponibles.

2 Conception

La phase de conception permet de réfléchir sur, comment le jeu sera construit avant de coder. L'idée principale est de créer un système composé de plusieurs parties qui fonctionnent de manière indépendante, c'est-à-dire, chaque élément du jeu, comme la carte, les combats et l'interface utilisateur, peut être développé et testé séparément sans affecter les autres parties du jeu. Cette façon de réfléxion est basée sur trois principes simples :

- **Séparation des éléments** : chaque fonctionnalité est séparée pour éviter qu'une modification ne casse pas le reste du programme (**flexible**).
- **Évolutivité** : la structure facilite les ajours futurs de nouveaux personnages ou de nouvelles mécaniques par exemple (**réutilisable**).

— **Flexibilité** : l'architecture permet une meilleure gestion de la carte générée dynamiquement (**évolutif**).
— **Clarté du code** : l'organisation permet de comprendre rapidement comment le programme fonctionne sans perdre du temps (**lisible** et **maintenable**).

Cela permet donc de maintenir et que les mécaniques de gameplay pourront être modifiées tout au long du développement du jeu.

2.1 Présentation du jeu

Présentation et principe du jeu

For The King est un jeu de rôle (RPG) et d'aventure intégrant des mécaniques de combat au tour par tour. Le joueur est placé dans le royaume, plongé dans le chaos. L'objetic est de préparer le héros à un affrontement contre le boss final pour restaurer la paix dans le royaume.

Le gameplay repose sur une boucle de progression stratégique autour de trois axes :

— **L'exploration et les quêtes** : Le joueur parcourt la carte pour découvrir des campements, des sanctuaires et remplir des objectifs secondaires.
— **Le combat au tour par tour** : Les affrontements opposent le personnage aux diffrérents monstres. Chaque tour permet de choisir entre une attaque légère (précise mais moins puissante) et une attaque lourde (dégâts élevés mais probabilité d'échec élevés).
— **La gestion des ressources** : L'élimination des ennemis octroie de l'or et des objets de soin. L'or est une ressource essentielle permettant de se reposer afin de regagner de la vie ou d'acheter des kits de soin dans les campements.

Mécaniques de progression

La progression du personnage est nécessaire pour vaincre le boss final. Le joueur peut obtenir des bénédictions via les sanctuaires. Les sanctuaires sont rares c'est pourquoi ils offrent des bonus de statistiques permanents. Chaque sanctuaire ne peut être utilisé qu'une seule fois, ce qui demande au joueur de faire des choix stratégiques durant son aventure. De plus, le personnage peut gagner de l'expérience en gagnant des combats contre des monstres. En gagnant de l'expérience, le personnage peut monter de niveau, ce qui augmente ses statistiques légèrement.

2.2 Fonctionnalités du jeu

Fonctionnalité	Explication
Génération aléatoire de la carte	Utilisation de l'algorithme Perlin Noise.
Personnage	Chaque personnage a une classe et des attributs.
Inventaire	Chaque personnage a un inventaire qui contient des objets.
Objet	On peut obtenir des objets en tuant des monstres.
Exemples d'objets	Kits de soins et pièces d'or.
Utilisation d'un objet	On peut utiliser tous les objets.
Interaction avec monstre	Cliquer la case du monstre sur la carte.
Options du combat	On peut soit fuir, soit attaquer.
Déroulement du combat	Tour par tour, commence par le personnage joueur.
Attaques possibles	Lourde ou légère.
Obstacles sur la carte	Montagnes, cactus, arbres etc.
Bâtiments sur la carte	Campement qui sert à regagner des PVs.
Sanctuaires sur la carte	Donnent un bonus si découverts et validés.
Expérience	Gagner un combat rapporte de l'EXP.
Niveau	Monter en niveau améliore les statistiques.
Quêtes	Finir les quêtes pour avoir des récompenses.

Table 1 – Ceci est un tableau présentant les différentes fonctionnalités du jeu et une explication.

3 Organisation du projet

3.1 Planning prévisionnel

Pour planifier les tâches, nous avons utilisé un outil spécifique : les diagrammes de Gantt. Cet outil nous a permis de créer un planning prévisionnel au début du projet, ainsi qu'un planning réel à la fin du projet. Cela nous a permis de comparer les deux pour voir si on avait respecté le planning prévisionnel initial.

1. D'abord on a fait ça...
2. Puis on a fait ceci...
3. Enfin on a fait cela.

3.2 Répartition des tâches

Tâche effectuée	Nom et prénom
Placement du personnage sur la carte	Durand Léo
Mécaniques de déplacement	Durand Léo
Création et utilisation des objets	Durand Léo
Système d'inventaire	Durand Léo
Sauvegarde des parties	Durand Léo
Affichage de la carte	Durand Léo
Tests Unitaires	Durand Léo
Boss final et son combat	Durand Léo
Musique	Durand Léo, Tsamarayev Massoud
Affichage des combats	Durand Léo, Tsamarayev Massoud
Génération de l'eau	Tsamarayev Massoud
Affichage hexagonal	Tsamarayev Massoud
Mécaniques du combat	Tsamarayev Massoud
Menu principal	Tsamarayev Massoud
Menu de pause	Tsamarayev Massoud
Graphismes	Tsamarayev Massoud
Système de niveau	Tsamarayev Massoud
Placement aléatoire des monstres	Tsamarayev Massoud
Quêtes	Tsamarayev Massoud
Interactions avec les sanctuaires	Tsamarayev Massoud
Interactions avec les campements	Tsamarayev Massoud
Documentation Doxygen	Mohamed Saandi
Génération des biomes	Mohamed Saandi
Interactions avec les monstres	Mohamed Saandi
Gantt prévisionnel	Mohamed Saandi
Bruitages	Mohamed Saandi
Création des monstres et leur attributs	Toute l'équipe
Création des personnages et leur attributs	Toute l'équipe
Génération de la carte	Toute l'équipe

Table 2 – Ceci est un tableau présentant les différents étudiants qui ont travaillé sur ce projet ainsi que les tâches qu'ils ont effectué.

Ici, je fais référence à mon tableau 3.2

3.3 Outils techniques utilisés

— Le langage C consiste en un langage de programmation bas niveau créé en 1972 par Dennis Ritchie et Ken Thompson. C'est le langage que nous utilisons dans ce projet.

— GCC est un compilateur très connu et utilisé pour transformer le code source en langage C en programme binaire exécutable.
— SDL2 est une librairie en C qui permet de créer des interfaces graphiques.
— Makefile sert à mettre des règles de compilation automatique, et ne recompile que les fichiers qui ont changé.
— Valgrind permet de vérifier les fuites de mémoire dans un programme binaire exécutable.
— Git est un gestionnaire de version, c'est-à-dire un système qui enregistre l'évolution d'un fichier ou d'un ensemble de fichiers au cours du temps de manière à ce que l'on puisse rappeler une version antérieure d'un fichier à tout moment (ProGit, S. Shacon, B. Straub).
— Doxygen est utilisé dans notre projet en tant que générateur de documentation, plus précisement c'est un outil de programmation qui crée de la documentation destinée aux programmeurs ou aux utilisateurs finaux. Le générateur se base sur les codes sources commentés et les fichiers binaires.
— Des diagrammes de Gantt sont utilisés pour planifier les tâches dans notre projet.
— LaTeX permet de formatter à l'aide de commandes un document, un rapport, un article etc. Il a été utilisé pour ce rapport.
— Des IA génératives comme ChatGPT et Gemini ont été utilisées pour générer les images, les textures et les sprites du jeu.
— Certains éditeurs d'image en ligne ont également servis à recadrer, redimensionner ou dessiner sur les images.
— Adobe Photoshop pour dessiner et modifier à partir des sprites générés par ChatGPT et Gemini.

4 Développement

Cette partie décrit en détail les aspects techniques du développement du projet. Nous présentons les principaux systèmes du jeu ainsi que leur implémentation en langage C, notamment la génération aléatoire de la carte, l'affichage du jeu avec SDL2, la gestion des combats, la gestion du menu, du personnage et des sauvegardes.

4.1 Génération aléatoire de la carte

La carte du jeu est un tableau en deux dimensions composée de structures de cases. Elle est crée par allocation dynamique pour qu'on puisse régler la taille de la carte, on le fait notamment dans les tests unitaires. Chaque case a ses propres caractéristiques :
— Le type de sanctuaire présent sur la case.
— Un entier représentant si le bonus du sanctuaire a été saisi.

— Le biome de la case : FORET, DESERT, NEIGE, EAU, TERRE.
— Le terrain ou l'obstacle de la case, comme des montagnes, des arbres, un cactus etc.
— Le bâtiment sur la case, si il existe, comme un campement par exemple.
— Un pointeur sur un monstre, si il existe.
— Un entier représentant si la case est visible ou couverte par le brouillard.

La génération de la carte est basée sur l'algorithme de Perlin Noise. L'avantage de cet algorithme est qu'il donne des résultats plus naturels que simplement un générateur de nombres aléatoires. Au lieu d'utiliser de l'aléatoire pur, le Perlin noise organise des valeurs sur une grille et les mélange doucement pour éviter les cassures. On obtient alors un bruit "fluide" et continu, qui ressemble à des formes naturelles.

4.2 Affichage du jeu

L'affichage du jeu est basé sur la grille hexagonales *flat-top*, cela signifie que le haut et le bas de l'hexagone sont plats. Chaque hexagone a trois données : le rayon, la largeur et la hauteur, qui sont calculées à partir de la taille de la case définie. Pour placer correctement chaque case, leurs coordonnées de grille sont converties en pixels. Les colonnes impaires sont décalées verticalement d'une demi-case vers le bas.

L'espacement horizontal entre deux colonnes est calculé de sorte que les hexagones s'emboîtent correctement sans trou visible. La caméra est centrée sur le personnage. À chaque image, on calcule la position du personnage en pixels, puis on fait un décalage global qu'on applique pour chaque case. Donc, chaque déplacement du personnage, implique le mouvement de la carte entière autour de lui et le personnage reste visuellement au centre de l'écran.

L'ordre du rendu suit les étapes suivantes : dessin des tuiles hexagonales avec leur biome, puis les éléments du décor tels que les obstacles et les bâtiments. Ensuite, le personnage est affiché par-dessus la carte et, finalement,

l'interface utilisateur : barre de vie, barre d'expérience, inventaire, panneaux de quêtes et menus. Toutes les dimensions et positions sont calculées dynamiquement via SDL2 en fonction de la taille réelle de la fenêtre, ce qui permet au jeu de s'adapter à n'importe quelle résolution sans déformation de l'affichage.

4.3 Gestion des combats

34

4.4 Gestion du menu

Le menu est la page principale. On peut effectuer plusieurs actions :

- **Nouvelle aventure** : Lorsqu'on clique sur le bouton "Nouvelle Aventure", un nouveau menu s'ouvre avec un bouton "+" qui permet de choisir un personnage. À chaque clic, le type de personnage change, ce qui permet de choisir le personnage souhaité
- **Continuer** : Le bouton "Continuer" permet de charger la partie sauvegardée auparavant depuis un fichier .txt
- **Options** : Le bouton "Options" permet de baisser ou augmenter le volume, et affiche également toutes les commandes du jeu
- **Quitter** : Lorsque le bouton "Quitter" est cliqué, un message s'affiche au centre de l'écran qui demande une confirmation, si l'utilisateur confirme, le jeu se ferme

Le menu est essentiel dans l'expérience utilisateur. Il a été implémenté avec la bibliothèque SDL2, qui permet de gérer les événements (clics, clavier) ainsi que le rendu graphique des éléments interactifs. Les animations au survol des boutons permettent d'améliorer le visuel du menu et ce qui rend agréable l'expérience utilisateur. Cette approche assure une navigation fluide et intéractive.

4.5 Gestion du personnage

Un personnage est défini par son type (*BRUTE, CHASSEUR, ASSASIN et MAGE*) qui est de type énuméré. Chaque personnage possède des caractèristiques qui varient selon le type de personnage. Ces caractèristiques sont regroupés dans une structure qui permet de réprésenter le personnage. Le fonctionnement du personnage repose sur plusieurs étapes :

— **Initialisation et typage :** Le personnage est créé et configuré via la fonction *init_perso*. Cette fonction alloue la mémoire, définit les coordonnées de départ et applique un switch sur le type choisi pour attribuer les statistiques de base (force, intelligence, etc.) propres à chaque personnage.

— **Position sur la carte :**

Les coordonnées du personnage, représentées par deux entiers x et y, permettent de connaître la position du joueur sur la carte. Ensuite, la fonction *afficher_personnage* transforme ces coordonnées en pixels pour dessiner le joueur au bon endroit sur l'écran et s'assure que la vue reste bien centrée sur lui.

— **Statistiques et combat :**

Des entiers représentent les dégâts, la force, la santé actuelle (*sante*), le maximum de points de vie (*sante_max*), l'intelligence, la rapidité et le pourcentage d'éviter un coup (*evasion*). En combat, ces valeurs sont utilisées par des fonctions comme *attaque_legere* ou *attaque_lourde* pour calculer l'issue du combat.

— **Gestion des tours et progression :**

Déplacements : Le joueur possède des points de déplacements (*pts_deplacements*). À chaque mouvement, ces points sont décrémentés. La fonction *restaurer_points_deplacements* est appelée à chaque début de tour pour réinitialiser ces points de façon aléatoire selon la classe. La valeur actuelle peut être consultée via *get_pers_movements_points*.

Progression : Le niveau évolue avec l'expérience accumulée (*exp*). En cas de victoire, la fonction *monstre_tue* (appelée dans *combat_termine*) gère l'attribution de l'expérience et l'incrémentation du compteur *nb_victime*.

Économie : La variable *pieces* permet l'achat d'objets dans l'inventaire, lequel est initialisé par *initialiser_inventaire* lors de la création du personnage.

— **Libération de la mémoire :** Lorsque la partie se termine ou que le personnage a perdu, la fonction *detruire_perso* assure la libération de la structure et de son inventaire associé pour éviter toute fuite mémoire.

4.6 Gestion de la sauvegarde

Avant que le programme se termine, une sauvegarde de la partie est effectuée. Il est donc conseillé de terminer le programme proprement, en appuyant sur le bouton "Quitter" du menu principal. Cette sauvegarde contient les données du personnage, ainsi que la carte et les différents bâtiments et monstres qui se trouvent sur chaque case. Lorsqu'on relance le jeu à nouveau, et qu'on appuie sur le bouton "Continuer", le jeu charge la dernière et unique sauvegarde. Au lieu de générer une nouvelle carte, le jeu va récupérer les informations de l'ancienne partie. Les sauvegardes sont stockées dans un fichier .txt sous format binaire. A chaque sauvegarde, on écrase l'ancien fichier, et donc le nouveau fichier commence vide. D'abord, il faut sauvegarder la carte. Comme chaque case a les mêmes attributs et la même structure en mémoire, la sauvegarde de la carte devient simple. Il suffit de faire une boucle sur toutes les cases de la carte et de stocker dans le fichier tous les attributs à la suite.

5 Bilan et résultats

Le bilan de notre projet est assez positif. En effet, les fonctionnalités principales de notre jeu ont été réalisées. Malgré tout, plusieurs fonctionnalités secondaires n'ont pas pu être implementé, comme par exemple les équipements qui augmentent les statistiques du joueur. Le planning prévisionnel quant à lui n'a pas été entièrement respecté. D'abord, nous avons souvent pris du retard sur certaines tâches à faire. De plus, nous sommes parfois revenus sur des fonctionnalités précédemment implémentées pour les améliorer ou les modifier. Les raisons qui pourraient expliquer cela sont tout d'abord des bugs imprévus qui retardent le développement, mais également une mauvaise planification initiale des tâches et de leur durée. Pour améliorer notre projet, si on devait le continuer, on pourrait améliorer les graphismes, rajouter des animations, rajouter des nouveaux objets, monstres, etc. Ce projet nous a apporté plusieurs choses et compétences intéressantes. D'abord, il nous a appris une nouvelle compétence technique : l'utilisation de SDL2. Il nous a également permis de renforcer nos connaissances techniques en algorithmique, et sur les outils techniques comme Git et le débogage. Concrètement, réaliser ce projet nous a aidé à appliquer les connaissances théoriques vues en cours pour un projet plus "pratique". Nous avons également appris à organiser, planifier et communiquer correctement pour faire un travail de groupe. Le projet nous a par exemple familiarisé avec les diagrammes de Gantt pour la planification des tâches. Il nous a aussi permis de connaître nos limites et comment les surmonter.

6 Bibliographie

Références

[1] https://fr.khanacademy.org/computing/computer-programming/programming-natural-simulations/programming-noise/a/perlin-noise

7 Annexe

aaaa